

CSA05 – Database Management Systems

CO4 Code Challenge Solutions

1. Heap File Organization – Insert, Delete, Sequential Scan

```
class HeapFile:
    def __init__(self):
        self.records = []

    def insert(self, record):
        self.records.append(record)
        print("Inserted:", record)

    def delete(self, key):
        for record in self.records:
            if record["id"] == key:
                self.records.remove(record)
                print("Deleted:", record)
                return
        print("Record not found")

    def sequential_scan(self):
        print("\nSequential Scan:")
        for record in self.records:
            print(record)

heap = HeapFile()

heap.insert({"id": 101, "name": "Arun"})
heap.insert({"id": 102, "name": "Bala"})
heap.insert({"id": 103, "name": "Charan"})

heap.sequential_scan()

heap.delete(102)

heap.sequential_scan()
```

2. Static Hashing with Chaining for Student Database

TABLE_SIZE = 10

```
class StaticHashing:
    def __init__(self):
        self.table = [[] for _ in range(TABLE_SIZE)]

    def hash_function(self, student_id):
        return student_id % TABLE_SIZE

    def insert(self, student_id, name):
        index = self.hash_function(student_id)
        self.table[index].append((student_id, name))

    def search(self, student_id):
        index = self.hash_function(student_id)

        for record in self.table[index]:
            if record[0] == student_id:
                return record

        return None

    def display(self):
        for i in range(TABLE_SIZE):
            print(i, "->", self.table[i])
```

```
db = StaticHashing()
```

```
# Simulation of student records
```

```
for i in range(1, 501):
    db.insert(i, "Student" + str(i))
```

```
db.display()
```

```
print("\nSearch Result:")
print(db.search(250))
```

3. Sorted File Organization with Binary Search

```
class SortedFile:
    def __init__(self):
```

```

        self.records = []

    def insert(self, key, name):
        self.records.append((key, name))
        self.records.sort()

    def binary_search(self, key):
        low = 0
        high = len(self.records) - 1

        while low <= high:
            mid = (low + high) // 2

            if self.records[mid][0] == key:
                return self.records[mid]

            elif self.records[mid][0] < key:
                low = mid + 1

            else:
                high = mid - 1

        return "Record not found"

    def display(self):
        print("Sorted Records:")
        for record in self.records:
            print(record)

sf = SortedFile()

sf.insert(105, "Arun")
sf.insert(101, "Bala")
sf.insert(103, "Charan")
sf.insert(102, "David")

sf.display()

print("\nSearch Result:")
print(sf.binary_search(103))

```

4. B-Tree of Order 4 – Insert, Search and Display

```

class BTreeNode:
    def __init__(self, leaf=False):
        self.keys = []
        self.children = []
        self.leaf = leaf

class BTree:
    def __init__(self):
        self.root = BTreeNode(True)

    def search(self, node, key):
        i = 0

        while i < len(node.keys) and key > node.keys[i]:
            i += 1

        if i < len(node.keys) and node.keys[i] == key:
            return True

        if node.leaf:
            return False

        return self.search(node.children[i], key)

    def insert(self, key):
        root = self.root

        if len(root.keys) == 3:
            new_root = BTreeNode()
            new_root.children.append(root)
            self.split_child(new_root, 0)
            self.root = new_root
            self.insert_non_full(new_root, key)
        else:
            self.insert_non_full(root, key)

    def insert_non_full(self, node, key):
        i = len(node.keys) - 1

        if node.leaf:
            node.keys.append(0)

            while i >= 0 and key < node.keys[i]:

```

```

        node.keys[i + 1] = node.keys[i]
        i -= 1

    node.keys[i + 1] = key

else:
    while i >= 0 and key < node.keys[i]:
        i -= 1

    i += 1

    if len(node.children[i].keys) == 3:
        self.split_child(node, i)

    if key > node.keys[i]:
        i += 1

    self.insert_non_full(node.children[i], key)

def split_child(self, parent, index):
    full_node = parent.children[index]
    new_node = BTreeNode(full_node.leaf)

    middle_key = full_node.keys[1]

    new_node.keys = full_node.keys[2:]
    full_node.keys = full_node.keys[:1]

    if not full_node.leaf:
        new_node.children = full_node.children[2:]
        full_node.children = full_node.children[:2]

    parent.children.insert(index + 1, new_node)
    parent.keys.insert(index, middle_key)

def display(self, node=None, level=0):
    if node is None:
        node = self.root

    print("Level", level, ":", node.keys)

    for child in node.children:
        self.display(child, level + 1)

```

```

btree = BTree()

keys = [10, 20, 5, 6, 12, 30, 7, 17]

for key in keys:
    btree.insert(key)

print("B-Tree:")
btree.display()

print("\nSearch for 12:", btree.search(btree.root, 12))
print("Search for 25:", btree.search(btree.root, 25))

```

5. B+ Tree with Leaf-Level Linking and Range Query

```

class LeafNode:
    def __init__(self):
        self.keys = []
        self.next = None

class SimpleBPlusTree:
    def __init__(self, max_keys=3):
        self.max_keys = max_keys
        self.leaves = [LeafNode()]

    def insert(self, key):
        leaf = self.leaves[-1]

        # Find correct leaf
        for node in self.leaves:
            if not node.keys or key <= node.keys[-1]:
                leaf = node
                break

        leaf.keys.append(key)
        leaf.keys.sort()

        # Split leaf when overflow occurs
        if len(leaf.keys) > self.max_keys:
            mid = len(leaf.keys) // 2

            new_leaf = LeafNode()

```

```

        new_leaf.keys = leaf.keys[mid:]
        leaf.keys = leaf.keys[:mid]

        new_leaf.next = leaf.next
        leaf.next = new_leaf

        index = self.leaves.index(leaf)
        self.leaves.insert(index + 1, new_leaf)

def display_leaves(self):
    current = self.leaves[0]

    print("Leaf Level:")
    while current:
        print(current.keys, end=" -> ")
        current = current.next

    print("None")

def range_query(self, start, end):
    result = []

    current = self.leaves[0]

    while current:
        for key in current.keys:
            if start <= key <= end:
                result.append(key)
        current = current.next

    return result

tree = SimpleBPlusTree()

keys = [10, 20, 5, 6, 12, 30, 7, 17, 25, 35]

for key in keys:
    tree.insert(key)

tree.display_leaves()

print("Range 10 to 30:")
print(tree.range_query(10, 30))

```

6. Dense Primary Index on a Sorted File

```
records = [
    (101, "Arun"),
    (102, "Bala"),
    (103, "Charan"),
    (104, "David"),
    (105, "Eshan")
]

# Dense index: Every record has an index entry
dense_index = {}

for position, record in enumerate(records):
    key = record[0]
    dense_index[key] = position

def display_index():
    print("Dense Primary Index")
    print("Key\tRecord Position")

    for key, position in dense_index.items():
        print(key, "\t", position)

def search(key):
    if key in dense_index:
        position = dense_index[key]
        return records[position]
    return "Record not found"

display_index()

print("\nSearch Result:")
print(search(103))
```

7. Extendible Hashing with Directory Doubling

```
class ExtendibleHashing:
    def __init__(self, bucket_size=2):
```

```

self.global_depth = 1
self.bucket_size = bucket_size

self.directory = {
    "0": [],
    "1": []
}

def hash_key(self, key):
    return bin(key)[2:].zfill(8)

def get_index(self, key):
    binary = self.hash_key(key)
    return binary[-self.global_depth:]

def double_directory(self):
    print("\nDirectory Doubling Occurred!")

    old_directory = self.directory
    self.global_depth += 1
    self.directory = {}

    for index, bucket in old_directory.items():
        self.directory["0" + index] = bucket.copy()
        self.directory["1" + index] = bucket.copy()

def insert(self, key):
    while True:
        index = self.get_index(key)

        if index not in self.directory:
            self.directory[index] = []

        bucket = self.directory[index]

        if len(bucket) < self.bucket_size:
            bucket.append(key)
            print("Inserted", key, "in bucket", index)
            break
        else:
            self.double_directory()

def display(self):
    print("\nGlobal Depth:", self.global_depth)

```

```

print("Directory:")

for index, bucket in self.directory.items():
    print(index, "->", bucket)

eh = ExtendibleHashing(bucket_size=2)

keys = [1, 2, 3, 4, 5, 6, 7, 8]

for key in keys:
    eh.insert(key)

eh.display()

```

8. Secondary Storage Block Access – Sequential vs Indexed Retrieval

```
import math
```

```

TOTAL_RECORDS = 1000
RECORDS_PER_BLOCK = 10

```

```
TOTAL_BLOCKS = math.ceil(TOTAL_RECORDS / RECORDS_PER_BLOCK)
```

```

print("Total Records:", TOTAL_RECORDS)
print("Records Per Block:", RECORDS_PER_BLOCK)
print("Total Blocks:", TOTAL_BLOCKS)

```

```

# Worst-case sequential search
sequential_io = TOTAL_BLOCKS

```

```

# Indexed retrieval:
# Assume binary search on index blocks + 1 data block
index_io = math.ceil(math.log2(TOTAL_BLOCKS))
data_io = 1

```

```
indexed_io = index_io + data_io
```

```

print("\nSequential Retrieval:")
print("Number of I/O Operations =", sequential_io)

```

```

print("\nIndexed Retrieval:")
print("Index Block I/O =", index_io)
print("Data Block I/O =", data_io)

```

```
print("Total I/O Operations =", indexed_io)
```

```
print("\nComparison:")
```

```
print("Sequential I/O:", sequential_io)
```

```
print("Indexed I/O:", indexed_io)
```

9. Sparse Index vs Dense Index with Timing Analysis

```
import time
```

```
# Create sorted file
```

```
records = [(i, "Student" + str(i)) for i in range(1, 10001)]
```

```
# Dense Index
```

```
dense_index = {}
```

```
for position, record in enumerate(records):
```

```
    dense_index[record[0]] = position
```

```
# Sparse Index: One entry for every 100 records
```

```
sparse_index = {}
```

```
BLOCK_SIZE = 100
```

```
for position in range(0, len(records), BLOCK_SIZE):
```

```
    sparse_index[records[position][0]] = position
```

```
def dense_search(key):
```

```
    if key in dense_index:
```

```
        return records[dense_index[key]]
```

```
    return None
```

```
def sparse_search(key):
```

```
    suitable_position = 0
```

```
    for index_key in sparse_index:
```

```
        if index_key <= key:
```

```
            suitable_position = sparse_index[index_key]
```

```
        else:
```

```
            break
```

```

    for i in range(suitable_position,
                   min(suitable_position + BLOCK_SIZE, len(records))):
        if records[i][0] == key:
            return records[i]

    return None

key = 8765

# Dense index timing
start = time.perf_counter()
result1 = dense_search(key)
end = time.perf_counter()

dense_time = end - start

# Sparse index timing
start = time.perf_counter()
result2 = sparse_search(key)
end = time.perf_counter()

sparse_time = end - start

print("Dense Search Result:", result1)
print("Dense Index Time:", dense_time)

print("\nSparse Search Result:", result2)
print("Sparse Index Time:", sparse_time)

print("\nPerformance Comparison:")
if dense_time < sparse_time:
    print("Dense Index is faster for direct record retrieval.")
else:
    print("Sparse Index was faster in this execution.")

```

10. B+ Tree Construction, Traversal and Range Query (15 to 45)

```

class BPlusLeaf:
    def __init__(self):
        self.keys = []
        self.next = None

```

```

class BPlusTree:
    def __init__(self, order=4):
        self.order = order
        self.leaves = [BPlusLeaf()]

    def insert(self, key):
        target = self.leaves[-1]

        for leaf in self.leaves:
            if len(leaf.keys) == 0 or key <= leaf.keys[-1]:
                target = leaf
                break

        target.keys.append(key)
        target.keys.sort()

        # Split when number of keys reaches order
        if len(target.keys) >= self.order:
            middle = len(target.keys) // 2

            new_leaf = BPlusLeaf()
            new_leaf.keys = target.keys[middle:]
            target.keys = target.keys[:middle]

            new_leaf.next = target.next
            target.next = new_leaf

            position = self.leaves.index(target)
            self.leaves.insert(position + 1, new_leaf)

    def traverse(self):
        print("B+ Tree Leaf Traversal:")

        current = self.leaves[0]

        while current:
            for key in current.keys:
                print(key, end=" ")
            current = current.next

        print()

```

```

def range_query(self, start, end):
    result = []

    current = self.leaves[0]

    while current:
        for key in current.keys:
            if start <= key <= end:
                result.append(key)

        current = current.next

    return result

```

```

bplus = BPlusTree(order=4)

keys = [10, 20, 5, 15, 25, 30, 35, 40, 45, 50, 17, 22, 27, 42]

for key in keys:
    bplus.insert(key)

bplus.traverse()

print("\nRange Query: Keys between 15 and 45")
result = bplus.range_query(15, 45)

print("Matching Records:")
for key in result:
    print(key)

```

Expected matching records for Question 10:

```

15
17
20
22
25
27
30
35
40
42
45

```